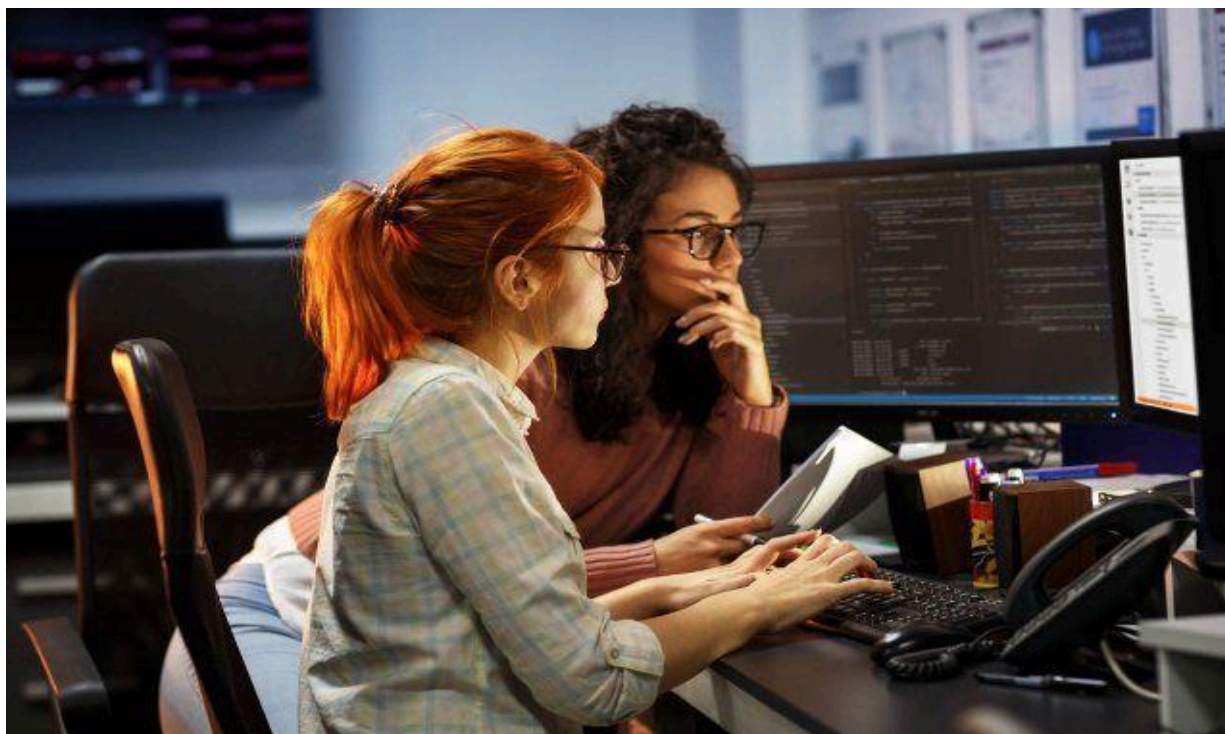




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Programación Orientada a Objetos	Unidad #: 1	Tema: Programación y Principios de Diseño Orientado a Objetos.
	SEMANA 3	
		Autor:

Introducción:

Bienvenidos al material de lectura #3 de la Unidad #1 de la materia Programación Orientada a Objetos. En este documento exploramos la introducción y los conceptos de clases, objetos, métodos y paso de argumentos a métodos.

Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en las actividades propuestas, ya que contribuirán significativamente a tu proceso de aprendizaje.

1.6 Clases y Objetos en Java

En el corazón de la Programación Orientada a Objetos (POO) y, por ende, de Java, se encuentran dos conceptos fundamentales: las **clases** y los **objetos**. Entender su diferencia es el primer paso para dominar este paradigma.

Piénsalo de esta manera: una **clase** es como un **plano** o un **molde** para crear algo. Por ejemplo, el plano de un coche. Este plano define todas las características y funcionalidades que tendrá cualquier coche que se fabrique a partir de él: una marca, un modelo, un color, y la capacidad de arrancar, frenar, etc. El plano en sí no es un coche; es solo la descripción detallada de cómo será uno.

Un **objeto**, por otro lado, es la **construcción real** hecha a partir de ese plano. Siguiendo nuestro ejemplo, cada coche individual que se fabrica usando el plano es un objeto. Puedes construir muchos coches (objetos) a partir de un solo plano (clase). Cada coche tendrá sus propias características específicas (quizás uno es un Toyota rojo y otro un Ford negro) pero todos compartirán la estructura y funcionalidades definidas en el plano.

En resumen:

- **Clase:** Es la plantilla, el molde o la definición abstracta. Define un conjunto de **atributos** (también llamados campos o variables de instancia) y **métodos** (comportamientos o acciones). No ocupa espacio en la memoria en tiempo de ejecución (aparte del necesario para su definición).
- **Objeto:** Es una **instancia** de una clase. Es una entidad concreta que existe en la memoria del programa y tiene un estado específico (valores para sus atributos). Puedes crear múltiples objetos a partir de una única clase, y cada uno será independiente de los demás.

**¿Te ha gustado lo que
has leído hasta ahora?**

Anota a continuación los aspectos que consideras más importantes de esta sección:

[illegible]

1.6.1. Implementación de Clases en Java

Implementar una clase en Java significa escribir el código que define esa plantilla. Se usa la palabra clave `class`. Por convención, los nombres de las clases en Java comienzan con una letra mayúscula.

Veamos un ejemplo de una clase `Coche`:

```
// El archivo debe llamarse Coche.java

public class Coche {

    // Atributos o campos (variables de instancia)

    String marca;

    String modelo;

    String color;

    boolean encendido; // Por defecto, para un boolean es
false

    // El método constructor. Se llama al crear un nuevo
objeto con 'new'

    public Coche(String marca, String modelo, String color) {

        // 'this' se usa para diferenciar el atributo de la
clase del parámetro del método

        this.marca = marca;

        this.modelo = modelo;

        this.color = color;

        this.encendido = false; // Inicializamos el coche
como apagado
    }
}
```

```
}

// Métodos (comportamientos)

public void arrancar() {

    this.encendido = true;

    System.out.println("El " + this.marca + " " +
this.modelo + " ha arrancado.");

}

public void apagar() {

    this.encendido = false;

    System.out.println("El " + this.marca + " " +
this.modelo + " se ha apagado.");

}

public void describir() {

    String estado = this.encendido ? "encendido" :
"apagado";

    System.out.println("Este es un " + this.marca + " " +
this.modelo + " de color " + this.color + " y está " + estado
+ ".");

}

}
```

En este código:

- `public class Coche` define una **clase** pública llamada `Coche`.
- `marca`, `modelo`, `color` y `encendido` son los **atributos**.
- `public Coche(...)` es el **constructor** de la clase. Se ejecuta automáticamente cuando se crea un objeto y se usa para inicializar sus atributos.
- `arrancar()`, `apagar()` y `describir()` son los **métodos**.

1.6.2. Declaración e Instanciación de Objetos en Java

Una vez que tienes la clase (el plano), puedes crear objetos (las construcciones). Este proceso se llama **instanciación** y en Java se realiza usando la palabra clave `new`.

El proceso consta de dos pasos:

1. **Declaración:** Se crea una variable que contendrá la referencia al objeto.
2. **Instanciación:** Se usa `new` para crear el objeto en memoria y se asigna su referencia a la variable declarada.

// Este código iría en un método main, por ejemplo, en otra clase.

```
public class Concesionario {  
    public static void main(String[] args) {  
        // Declaración e instanciación de objetos de la clase  
        Coche  
  
        Coche miCoche = new Coche("Toyota", "Corolla",  
        "Rojo");  
  
        Coche cocheDeAna = new Coche("Ford", "Mustang",  
        "Negro");  
  
        // Ahora miCoche y cocheDeAna son dos objetos  
        distintos e independientes
```

```
// Podemos llamar a sus métodos usando el operador  
punto (.)
```

```
miCoche.describir(); // Salida: Este es un Toyota  
Corolla de color Rojo y está apagado.
```

```
cocheDeAna.describir(); // Salida: Este es un Ford  
Mustang de color Negro y está apagado.
```

```
// Usemos un método para cambiar el estado de uno de  
los objetos
```

```
miCoche.arrancar(); // Salida: El Toyota Corolla ha  
arrancado.
```

```
// Verifiquemos el estado de ambos coches de nuevo  
  
miCoche.describir(); // Salida: Este es un Toyota  
Corolla de color Rojo y está encendido.
```

```
cocheDeAna.describir(); // Salida: Este es un Ford  
Mustang de color Negro y está apagado.
```

```
    }  
  
}
```

Como puedes ver, `miCoche` y `cocheDeAna` son dos objetos de la misma clase `Coche`, pero cada uno tiene su propio estado (sus propios valores para los atributos) y funcionan de manera independiente.

1.7. Paso de Parámetros a Métodos en Java

En Java, el paso de parámetros a métodos es un tema clave y a la vez simple: **Java siempre utiliza el paso por valor**. Sin embargo, es crucial entender qué significa esto para los tipos de datos primitivos y para los objetos.

Paso por Valor con Tipos Primitivos

Cuando pasas una variable de un tipo primitivo (`int`, `double`, `float`, `char`, `boolean`, etc.) a un método, se pasa una **copia** del valor de esa variable. Cualquier modificación que el método haga a ese parámetro **no afectará a la variable original** fuera del método.

Ejemplo:

```
public class PruebasPasoParametros {  
  
    public static void modificarNumero(int numero) {  
        numero = 100; // Se modifica la COPIA del valor  
        System.out.println("Dentro del método, el número es:  
" + numero); // Imprime 100  
    }  
  
    public static void main(String[] args) {  
        int miNumero = 10;  
        System.out.println("Antes de llamar al método,  
miNumero es: " + miNumero); // Imprime 10  
        modificarNumero(miNumero);  
        System.out.println("Después de llamar al método,  
miNumero es: " + miNumero); // Sigue imprimiendo 10  
    }  
}
```

}

Paso por Valor con Objetos (Paso por Valor de la Referencia)

Aquí es donde suele surgir la confusión. Cuando pasas un objeto a un método, lo que se pasa por valor es la **referencia** (la dirección de memoria) al objeto. Es decir, se hace una **copia del valor de la referencia**.

Esto tiene una consecuencia muy importante: tanto la variable original como el parámetro del método apuntan al **mismo objeto** en la memoria.

- Si usas esa referencia copiada para **modificar el estado interno del objeto** (cambiar el valor de sus atributos), los cambios **serán permanentes** y se verán reflejados fuera del método.
- Sin embargo, si dentro del método intentas que el parámetro apunte a un objeto completamente nuevo (`parametro = new Objeto();`), la variable original fuera del método **no se verá afectada**, ya que solo cambiaste a dónde apunta la copia de la referencia.

Ejemplo:

```
public class PruebasPasoParametros {  
  
    // Se usa la clase Coche definida anteriormente  
  
    public static void pintarCoche(Coche coche) {  
  
        // Se modifica un atributo del objeto original a  
        // través de la referencia  
  
        coche.color = "Azul";  
  
        System.out.println("Dentro del método, el color es: "  
+ coche.color); // Imprime Azul  
  
    }  
  
    public static void main(String[] args) {
```



```
        Coche miCoche = new Coche("Nissan", "Sentra",  
        "Gris");  
  
        System.out.println("Antes de llamar al método, el  
        color es: " + miCoche.color); // Imprime Gris  
  
        pintarCoche(miCoche);  
  
        System.out.println("Después de llamar al método, el  
        color es: " + miCoche.color); // Imprime Azul  
    }  
}
```

Paso de Parámetros entre Métodos

El paso de parámetros no se limita a la llamada inicial. Un método puede, a su vez, llamar a otro método y pasarle parámetros. Las reglas del paso por valor se aplican de la misma manera en cada una de estas llamadas anidadas.

Ejemplo en Java:

```
class Calculadora {  
  
    public int sumarYDuplicar(int a, int b) {  
        // 1. Llama al método sumar, pasándole copias de los  
        valores de 'a' y 'b'.  
  
        int suma = this.sumar(a, b);  
  
        // 2. Llama al método duplicar, pasándole una copia  
        del valor de 'suma'.  
    }  
}
```

```
        int resultadoFinal = this.duplicar(suma);

        return resultadoFinal;
    }

    private int sumar(int x, int y) {
        return x + y;
    }

    private int duplicar(int numero) {
        return numero * 2;
    }

    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        int resultado = calc.sumarYDuplicar(5, 3);

        // Flujo:

        // - sumarYDuplicar(5, 3) llama a sumar(5, 3) que
        devuelve 8.

        // - La variable local 'suma' vale 8.

        // - sumarYDuplicar llama a duplicar(8) que devuelve
        16.

        // - Se retorna 16.
```

```
        System.out.println("El resultado final es: " +  
resultado); // Salida: El resultado final es: 16  
    }  
}
```

En este flujo, el valor de retorno de un método (**sumar**) se convierte en el parámetro de entrada para otro método (**duplicar**), demostrando cómo la información fluye a través de diferentes partes de una clase, siempre siguiendo la regla del **paso por valor**.

